

Project 2: Data Classification Using AI

DecodeLabs · Artificial Intelligence Track · Batch 2026

Goal: Build a supervised classification model that identifies the species of an iris flower (*Setosa*, *Versicolor*, *Virginica*) from four measurements.

Pipeline (IPO framework):

| Input | Process | Output |

| Iris dataset, feature scaling | Train/test split, K-Nearest Neighbors | Confusion matrix, F1 score |

1. Setup

2. Load and understand the dataset

The Iris benchmark ships with scikit-learn, so no file upload is needed.

Samples: 150 | Features: 4 | Classes: 3

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
<class 'pandas.DataFrame'>
```

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal_length	150 non-null	float64
1	sepal_width	150 non-null	float64
2	petal_length	150 non-null	float64
3	petal_width	150 non-null	float64
4	species	150 non-null	str

dtypes: float64(4), str(1)

memory usage: 6.0 KB

Missing values: 0

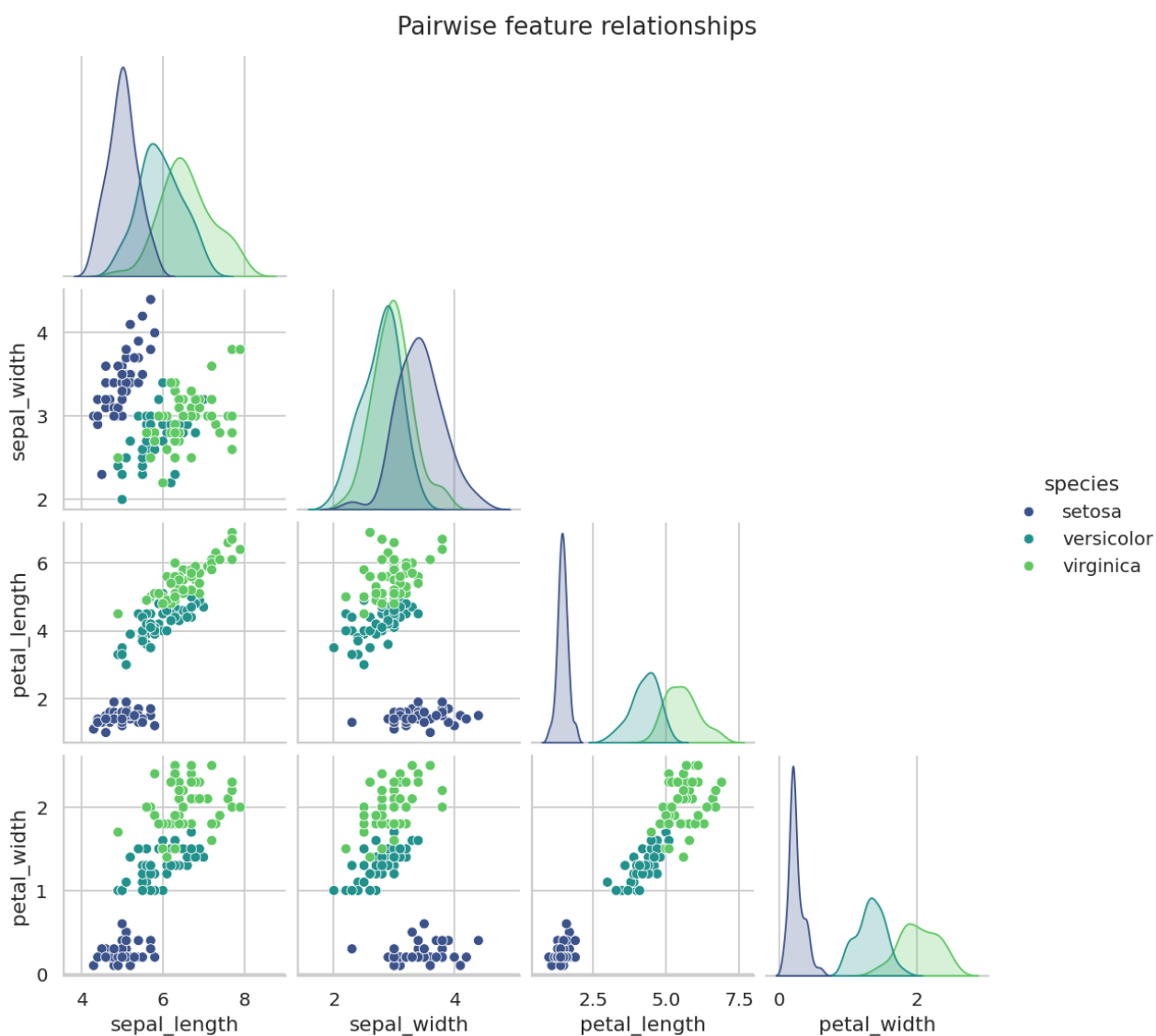
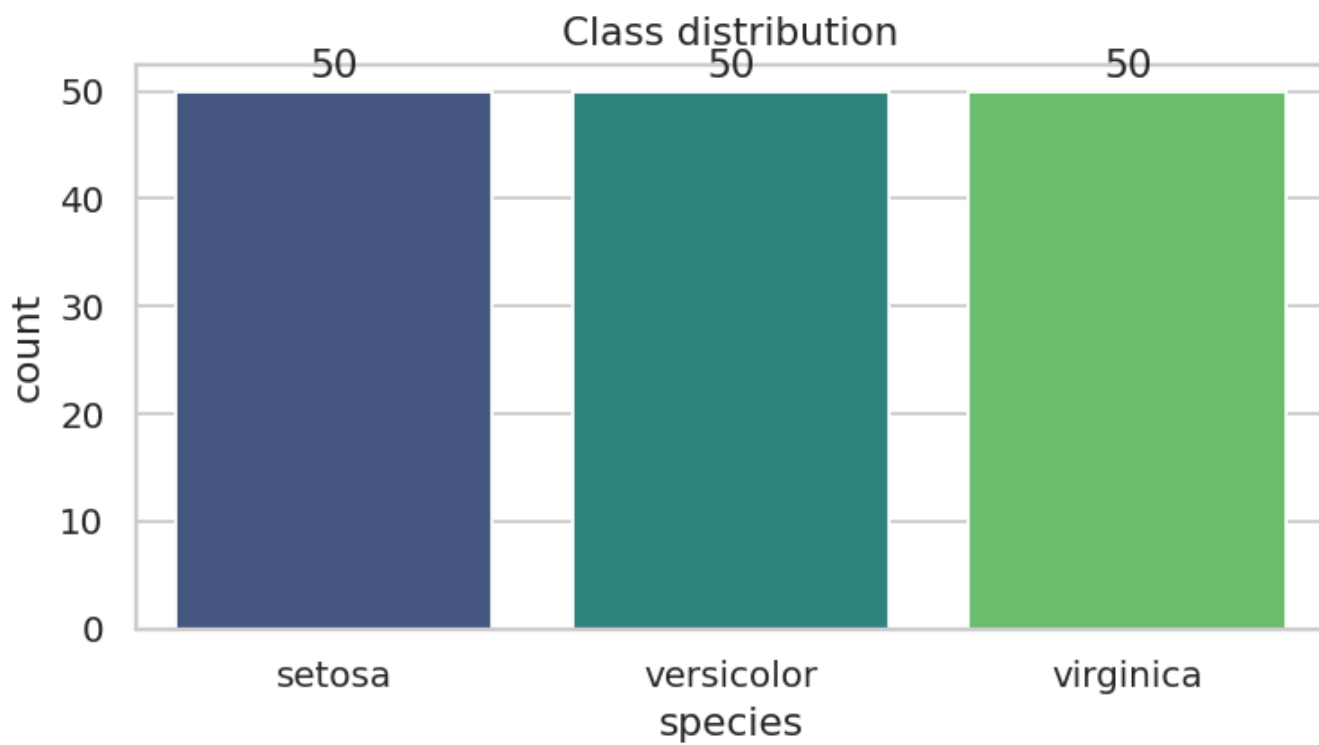
Duplicate rows: 1

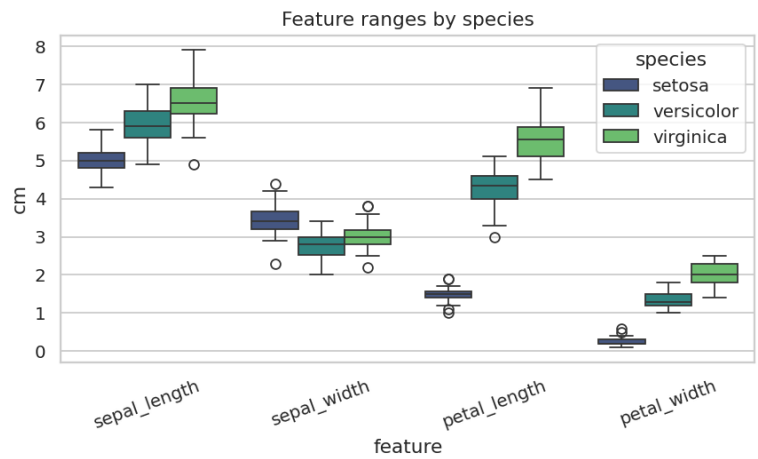
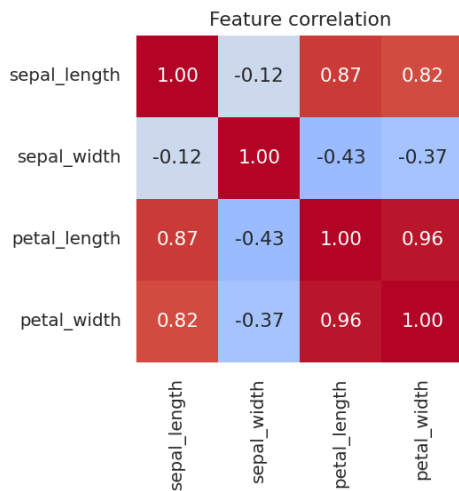
	count	mean	std	min	25%	50%	75%	max
sepal_length	150.0	5.84	0.83	4.3	5.1	5.80	6.4	7.9
sepal_width	150.0	3.06	0.44	2.0	2.8	3.00	3.3	4.4
petal_length	150.0	3.76	1.77	1.0	1.6	4.35	5.1	6.9
petal_width	150.0	1.20	0.76	0.1	0.3	1.30	1.8	2.5

Dataset summary: 150 samples, 4 numeric features (in cm) and 3 balanced classes with 50 samples each.

There are no missing values, so no cleaning is required. The single duplicate row is a genuine repeated measurement and is kept.

Exploratory analysis





Observations

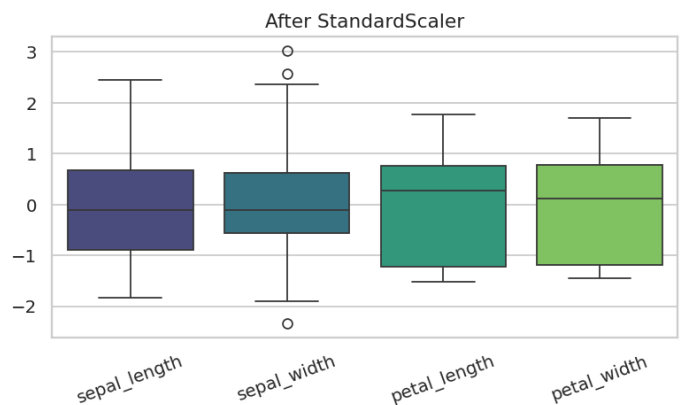
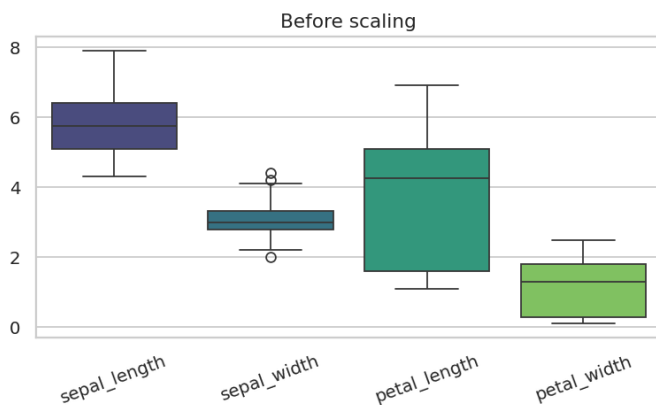
- *Setosa* is clearly separable from the other two species, mainly through its small petals.
- *Versicolor* and *Virginica* overlap a little, so these two are where classification errors are most likely.
- Petal length and petal width are the most informative features and are strongly correlated with each other.

3. Feature scaling and train / test split

- **Shuffle before splitting** to remove order bias. The raw Iris file is sorted by species.
- **Stratify** so each class keeps the same proportion in both sets.
- **StandardScaler** (mean = 0, variance = 1) is placed inside a Pipeline, so it is fitted on the training data only. This prevents data leakage from the test set.

Training set: 120 samples (80%)
Test set : 30 samples (20%)

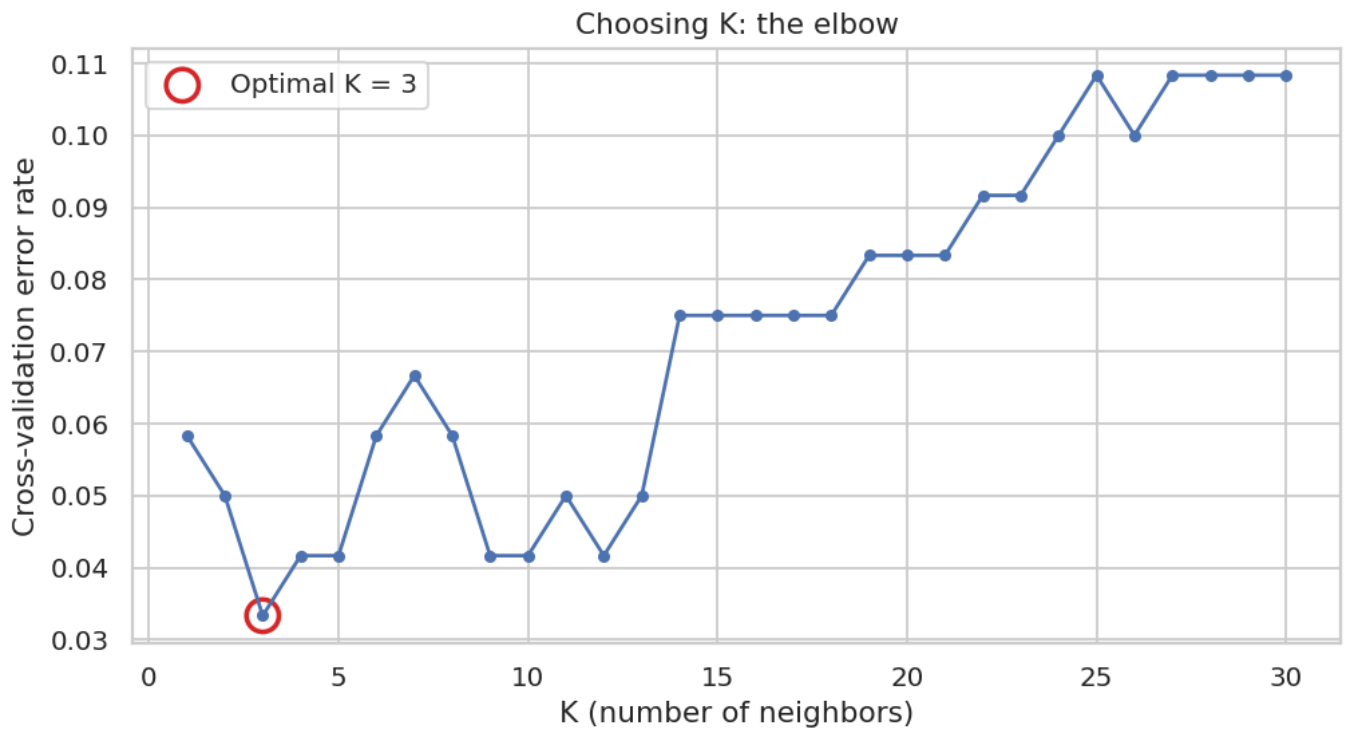
Test class counts:
species
setosa 10
virginica 10
versicolor 10



4. Tune the engine: choosing K

K-Nearest Neighbors classifies a flower by a majority vote among its K closest training samples. A very small K overfits to noise, and a very large K underfits. K is chosen using **5-fold cross-validation on the training set only**.

Optimal K = 3 (CV accuracy 0.967)



5. Build the model: Instantiate → Fit → Predict

Model trained.

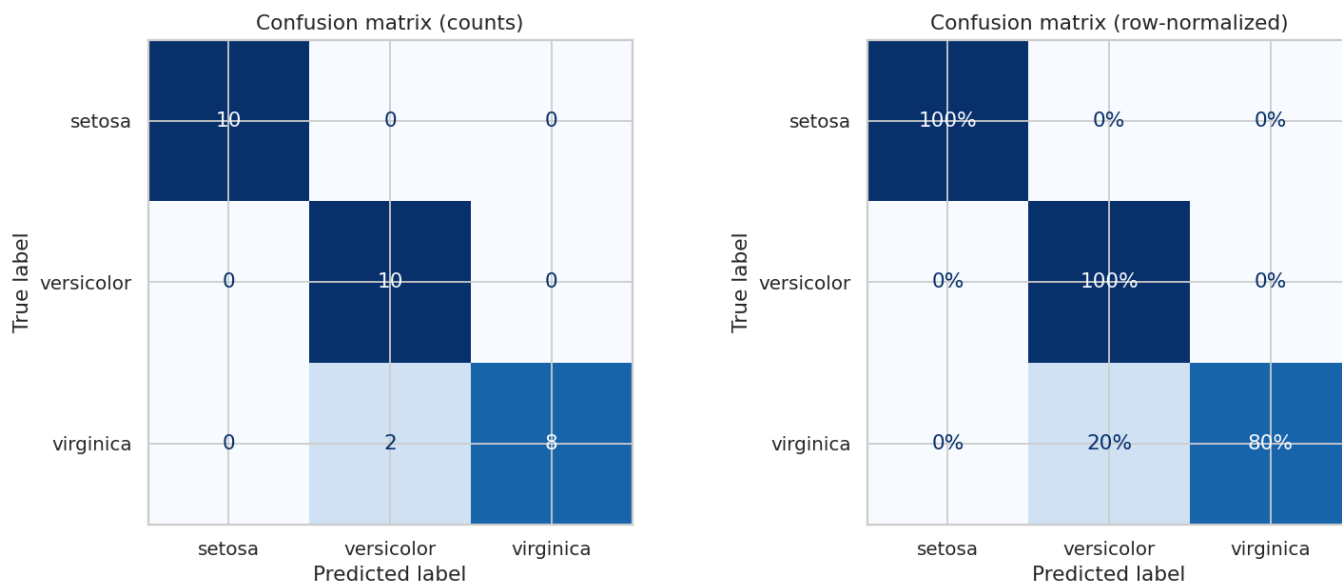
Sample predictions: ['setosa', 'virginica', 'versicolor', 'versicolor', 'setosa', 'versicolor', 'setosa', 'setosa']
 Actual labels : ['setosa', 'virginica', 'versicolor', 'versicolor', 'setosa', 'versicolor', 'setosa', 'setosa']

6. Output validation

Accuracy alone can be misleading, especially on imbalanced data. The confusion matrix, precision, recall and F1 give the full picture.

Score
 Metric
 Accuracy 0.933
 Precision (macro) 0.944
 Recall (macro) 0.933
 F1 score (macro) 0.933

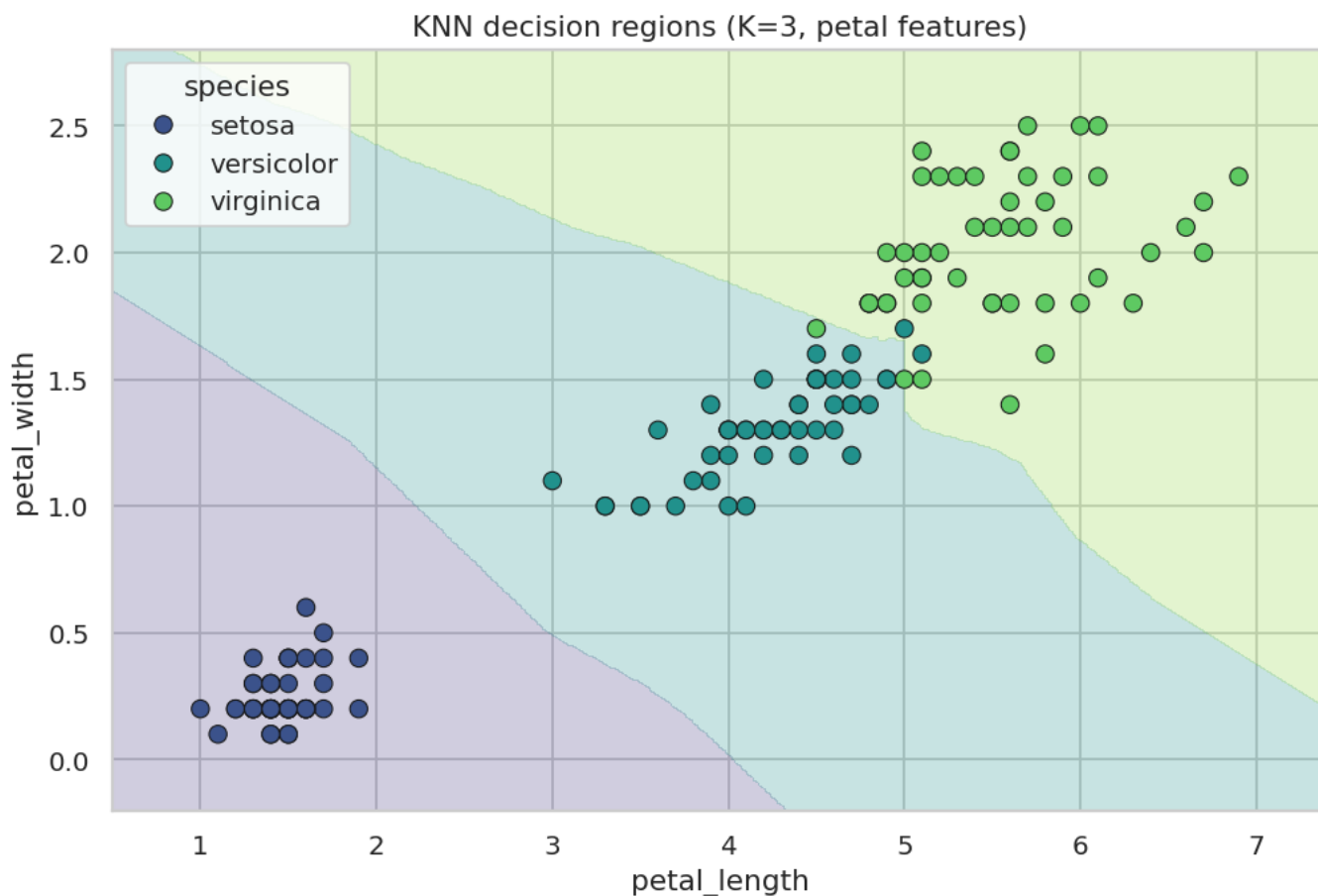
precision	recall	f1-score	support	
				setosa
	1.000	1.000	1.000	10
	0.833	1.000	0.909	10
	1.000	0.800	0.889	10
				accuracy
			0.933	30
	0.944	0.933	0.933	30
	0.944	0.933	0.933	30



Reading the matrix: the diagonal holds correct predictions. Any off-diagonal cell is a misclassification, and the *Versicolor / Virginica* pair is where these appear, as expected from the exploration plots.

7. Decision regions (2D view)

The model is retrained on two petal features so the decision boundaries can be drawn. This is for visualization only, and the model evaluated above uses all four features.

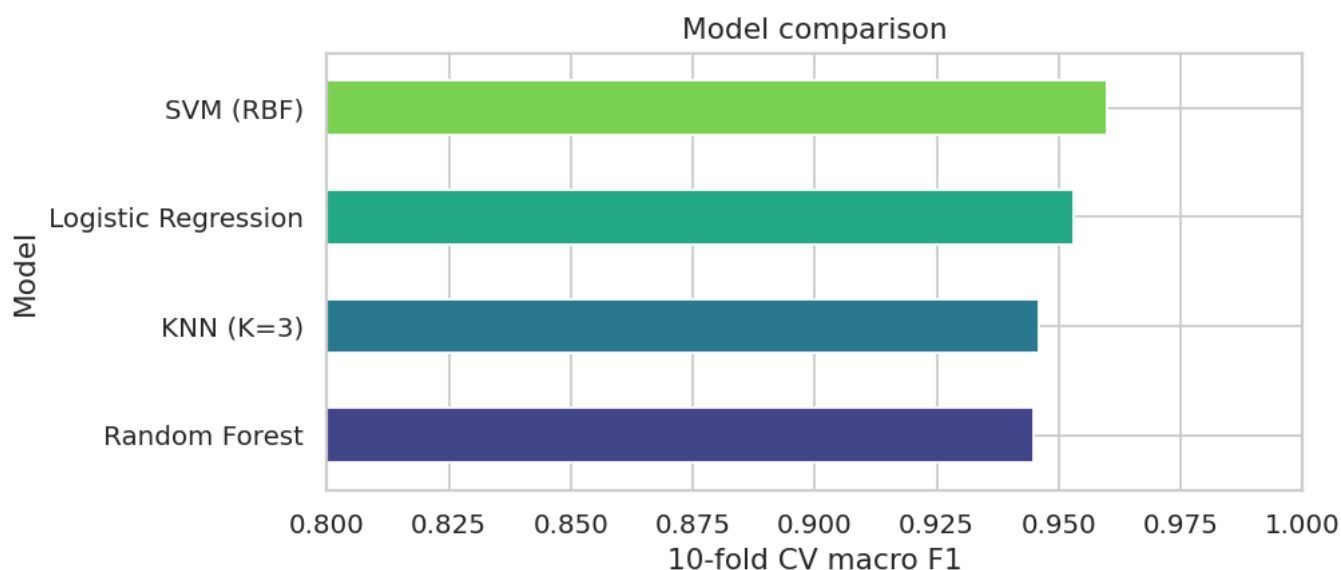


8. Robustness check and model comparison

A single 80/20 split can be lucky or unlucky. Here the KNN is re-checked with **10-fold cross-validation on the full dataset**, and compared with other standard classifiers.

CV Macro F1 (mean) CV Std Test Accuracy

Model			
KNN (K=3)	0.946	0.072	0.933
Logistic Regression	0.953	0.053	0.933
SVM (RBF)	0.960	0.044	0.967
Random Forest	0.945	0.060	0.900



9. Predict a new flower

```
Predicted species: versicolor
versicolor    100.0 %
setosa         0.0 %
virginica      0.0 %
dtype: str
```

10. Conclusion

```
Final model      : KNN (K=3) with StandardScaler
Test accuracy    : 93.3%
Test macro F1    : 0.933
10-fold CV F1    : 0.946 (+/- 0.072)
```

Summary

- **Pipeline:** the full workflow was completed: load → explore → scale → shuffled, stratified 80/20 split → tune K by cross-validation → fit → predict → validate.
- **Result:** KNN separates the three species with high accuracy. *Setosa* is classified almost perfectly, and the few errors involve *Versicolor* and *Virginica*, whose measurements overlap.
- **Rigor:** scaling happens inside a pipeline (no leakage), K is tuned without touching the test set, and results are confirmed with 10-fold cross-validation and compared with other classifiers.
- **Next step:** the manual points toward **Deep Learning and CNNs**, moving from tabular data to computer vision.